

Running This on an Offline (Air-Gapped) Machine — Debugging Guide

Your backend, frontend, and the LLM itself are already confirmed working. This doc is only about problems that show up **because the machine has no internet access**, specifically around `api.py` and the libraries it pulls in (`transformers`, `huggingface_hub`, `sentence-transformers`, `faiss`).

1. Installing packages in the first place

`pip install` needs PyPI, which won't work offline. Before you ship the machine:

1. On a connected machine, download all wheels:

```
bash

pip download -r requirements.txt -d ./wheels
```

2. Copy the `./wheels` folder to the offline machine.
3. Install from that folder instead of the internet:

```
bash

pip install --no-index --find-links=./wheels -r requirements.txt
```

If you skip this, nothing in this project even imports — this is the #1 thing to sanity check first.

2. Environment variables must be set BEFORE any Hugging Face import

`transformers`, `huggingface_hub`, and `sentence-transformers` all try to reach `huggingface.co` by default — to check for newer versions of a model, fetch configs, etc. On an offline machine this doesn't just fail, it can **hang for 10–30 seconds per attempt** while it times out, before finally erroring.

The fix is two environment variables, and they must be set **before the very first import of anything from** `transformers`/`huggingface_hub`/`sentence-transformers`/`langchain_huggingface` — Python imports run top-to-bottom, so if these are set even one line too late, the damage is already done for that import.

```
python
```

```
import os
os.environ.setdefault("HF_HUB_OFFLINE", "1")
os.environ.setdefault("TRANSFORMERS_OFFLINE", "1")
```

In `api.py` these are already the very first lines, before `fastapi` and before any `src.*` import — including `Embedding`, since sentence-transformer models are also fetched from the Hub. **Do not move the `src.*` imports above these two lines**, even accidentally during a refactor.

Optional extra safety net (older `huggingface_hub` versions sometimes still ping telemetry even in offline mode):

```
python
os.environ.setdefault("HF_HUB_DISABLE_TELEMETRY", "1")
```

3. Model folders must be 100% complete

With offline mode forced, `transformers` will *not* fall back to downloading a missing file — it just errors. Before deployment, check that `phi-3-mini`'s folder (and any embedding model folder) has everything a normal Hub repo would have, typically:

- `config.json`
- `generation_config.json`
- `tokenizer.json` / `tokenizer_config.json` / `tokenizer.model` (whichever the model uses)
- `model.safetensors` (or sharded `.safetensors` + `model.safetensors.index.json`)
- Any chat template file, if the model needs one for `ChatHuggingFace`

How to check quickly:

```
bash
ls -la ../rag_chatbot/LLM/phi-3-mini
```

Compare that file list against the model's page on huggingface.co (viewed on a connected machine beforehand) to make sure nothing's missing.

If a file is missing, the error usually looks like:

```
OSError: Can't load tokenizer for '...'. If you were trying to load it from
'https://huggingface.co/models', make sure you don't have a local directory
with the same name.
```

which is a bit misleading offline — it almost always means "a required file is missing from the local folder," not an actual network problem.

4. Relative paths depend on your working directory

`MODEL_PATH = "../rag_chatbot/LLM/phi-3-mini"` is relative. It only resolves correctly if you always launch `uvicorn` from the exact same folder. If DRDO's ops team starts the service via a systemd unit, a different shell, or a different working directory, this path silently points to the wrong place and you'll get a "file not found" style error that looks unrelated to offline-ness at first glance.

Safer version — make it relative to the file itself, not the shell's current directory:

```
python

import os
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE_DIR, "..", "rag_chatbot", "LLM", "phi-3-mini")
```

Same applies to `INDEX_PATH`/`METADATA_PATH` in `main.py` if those are also launched from a service manager rather than manually.

5. CPU vs GPU mismatches

If the DRDO machine has no GPU (common for air-gapped/secure machines):

- Make sure you installed `faiss-cpu`, not `faiss-gpu`.
 - Make sure your `torch` wheel is the CPU build, not a CUDA build — a CUDA wheel will still *import* but can throw confusing errors the moment it tries to touch a device, or just be extremely slow if it silently falls back.
 - `pipeline("text-generation", model=...)` in `llm.py` doesn't explicitly set `device=`, so it'll default sensibly, but it's worth confirming on the actual hardware once rather than assuming.
-

6. Quick smoke test before wiring up the frontend

Run this in a plain Python shell on the offline machine, with no other services running, to isolate whether a failure is "the model" or "the API":

```
python
```

```
import os
os.environ.setdefault("HF_HUB_OFFLINE", "1")
os.environ.setdefault("TRANSFORMERS_OFFLINE", "1")

from src.models.llm import LLM
llm = LLM(model_path="../rag_chatbot/LLM/phi-3-mini")
print(llm.generate("Say hello in one sentence."))
```

If this works but the FastAPI endpoint doesn't, the problem is in `api.py`/routing, not the model. If this fails, it's a model-loading issue — go back to steps 2 and 3.

7. Running the server (`uvicorn`)

`api.py` is a FastAPI app — it's run via **uvicorn**, the server that actually listens on a port and hands requests to the app:

```
bash

uvicorn api:app --host 0.0.0.0 --port 8000
```

- `api` = the filename `api.py` (no `.py`)
- `app` = the `FastAPI()` object defined inside it
- `--host 0.0.0.0` = accept connections from other machines on the network (use `127.0.0.1` if the frontend runs on this same machine only)
- `--port 8000` = pick whatever port the frontend is configured to hit

Offline-specific catch: `uvicorn` is not automatically installed with `fastapi` — it's a separate package. Make sure it's in your `requirements.txt` and included in the pre-downloaded wheels from Step 1, or this command will fail with `command not found` / `ModuleNotFoundError`.

If you want the server to survive a reboot or terminal closing on the actual deployed machine, run it as a systemd service or inside `tmux`/`screen` rather than a plain foreground terminal — a closed SSH session will kill a foreground `uvicorn` process.

8. What hasn't actually been verified (read this before assuming it's bulletproof)

Everything in this project was written/reviewed by reading code, not by running it end-to-end against your real model files and real `src/` modules. The folder structure has been confirmed to match (`chunking`), (`embedding`), (`ingestion/loader.py`), (`models/llm.py`),

(`retrieval/vector/faiss_retriever.py`), (`vectorstore/vectore_store.py`) — so import paths in (`api.py`) are correct. But a few things are still assumptions, not confirmed facts:

- **(`search_context`)'s signature.** (`api.py`) calls it as (`retriever.search_context(req.question, vectordb.index, vectordb.metadata, top_k=5)`). This is based on (`main.py`) calling (`search_context(query, vectordb.index, vectordb.metadata)`) (without (`top_k`)). If (`top_k`) isn't an actual accepted keyword argument in (`faiss_retriever.py`), this will raise a (`TypeError`) immediately on the first request. **Check (`faiss_retriever.py`)'s method signature to confirm.**
- **(`search_context`)'s return shape.** (`api.py`) assumes each result is a dict with a (`"text"`) key (`(r["text"] for r in results)`), based on (`retriever.print_results(query, results)`) in (`main.py`). If the actual return shape is different (e.g. a different key name, or objects instead of dicts), the context-building step will throw a (`KeyError`) or (`TypeError`). **Worth a quick look at (`faiss_retriever.py`) to confirm the key name.**
- **Whether (`llm.py`) still matches what was reviewed here.** The (`LLM`) class content was pasted into this chat once; if it's been edited since (different constructor args, different return shape from (`.generate()`)), (`api.py`)'s call to (`model.generate(prompt)`) and (`LLM(model_path=MODEL_PATH)`) need to still line up with the current file.
- **Completeness of the (`phi-3-mini`) model folder on the actual DRDO machine** — covered in Section 3, but worth repeating: this can't be checked from outside that machine.
- **Exact package versions installed offline** (`fastapi`), (`transformers`), (`langchain-huggingface`), (`torch`), (`faiss-cpu`) — version mismatches between what was tested on a connected dev machine and what ends up bundled for the offline machine can cause subtle failures that don't show up in code review at all.

None of this means the code is wrong — it means it hasn't been run against your real files, only reasoned about from what you've pasted/shown. The fastest way to close these gaps is to run the smoke test in Section 6, then hit (`/rag/query`) once with a real question and read the traceback if anything breaks — a real error message pinpoints the problem far faster than guessing from code alone.

Summary checklist before deployment

- ☐ All wheels pre-downloaded and installed with (`--no-index`)
- ☐ (`HF_HUB_OFFLINE`) / (`TRANSFORMERS_OFFLINE`) set as the very first lines of (`api.py`)
- ☐ Model folder(s) verified complete against the Hub listing
- ☐ Paths made absolute / anchored to (`__file__`), not the shell's cwd
- ☐ Confirmed (`faiss-cpu`) + CPU (`torch`) if there's no GPU
- ☐ Ran the standalone (`LLM`) smoke test successfully before testing through FastAPI
- ☐ (`uvicorn`) is installed (separate from (`fastapi`)) and included in the offline wheels

- ☐ Decided how the service is launched long-term (systemd/tmux, not a foreground SSH session)
- ☐ Confirmed `search_context()` actually accepts a `top_k` keyword argument
- ☐ Confirmed each result from `search_context()` has a `"text"` key
- ☐ Confirmed `llm.py` on disk still matches what `api.py` assumes (constructor args, `.generate()` return shape)